

# **DBMS PROJECT REPORT**

## **HELPDESKLITE- COMPLAINT MANAGEMENT SYSTEM**

### **TEAM MEMBERS :**

Abhai Krishnan  
Adithya Satheesh  
Adwaith K S  
Akshara Mohandas  
Akshaya Biju  
Amritha M R

## **ABSTRACT**

Helpdesk Lite is a lightweight, database-driven service request and complaint management system designed for small institutions, campuses, and residential communities. The system provides a structured platform for users to submit issues or service requests, track their status, and receive updates until resolution. It aims to replace informal and unorganized reporting methods with a centralized and transparent workflow. The application is built around a relational database model that manages users, requests, categories, and status updates efficiently. Core DBMS concepts such as entity–relationship modeling, primary and foreign key constraints, data integrity, and basic query operations are implemented to ensure consistency and reliability of data. The system supports essential operations including request creation, assignment, status tracking, and historical record maintenance. HelpDesk Lite is designed to be simple, scalable, and adaptable, allowing it to be extended with additional features such as role based access, analytics, and notification services in the future. By focusing on clarity, efficiency, and ease of use, the system demonstrates the practical application of database management principles in solving real-world service coordination problems.

## INTRODUCTION

In many organizations, service requests are often handled through informal methods such as verbal communication or scattered messaging platforms, leading to inefficiencies, lack of accountability, and poor tracking. HelpDesk Lite addresses this gap by providing a **centralized, structured, and transparent platform** for managing service requests efficiently.

The system is designed using a **relational database approach**, ensuring proper organization and integrity of data. It incorporates core DBMS concepts such as:

- Entity–Relationship modeling
- Primary and Foreign Key constraints
- Data consistency and normalization
- Efficient data retrieval using structured queries

The application is divided into multiple functional modules, including:

- **Authentication Module** for secure user access
- **Request Module** for submitting and tracking issues
- **Admin Module** for monitoring and managing requests
- **Assignment Module** for allocating tasks to responsible personnel
- **Status Module** for updating request progress

- **Database Module** for backend connectivity
- **User Interface Module** for an interactive experience

The system database consists of six main tables: **Users, Requests, Categories, Status, Assignments, and Request\_History**, which together ensure complete lifecycle tracking of each request from creation to resolution.

HelpDesk Lite is implemented using technologies such as **PHP, MySQL, HTML, and CSS**, making it simple, scalable, and easy to deploy. The system can be further enhanced with advanced features like role-based access control, notifications, and analytics dashboards.

Overall, this project demonstrates the **practical application of database management systems in solving real-world service coordination problems**, improving efficiency, transparency, and user experience.

## OBJECTIVES

To develop a **centralized system** for managing service requests and complaints efficiently

- To replace informal reporting methods with a **structured and transparent workflow**
- To enable users to **submit, track, and monitor** their requests in real time
- To provide administrators with tools to **manage, assign, and update requests effectively**
- To implement core **DBMS concepts** such as relational schema design, primary and foreign keys, and data integrity
- To maintain a complete **history of request status updates** for accountability and traceability
- To ensure **efficient data storage and retrieval** using structured queries
- To design a **user-friendly interface** for smooth interaction between users and administrators
- To support **scalability**, allowing future enhancements like notifications, analytics, and role-based access control
- To demonstrate the **real-world application of database systems** in solving service coordination problems

## **SYSTEM REQUIREMENTS**

Hardware Requirements

Laptop / PC

Internet connection

Software Requirements

XAMPP Server

PHP

MySQL

VS Code

Web Browser

## **SYSTEM ARCHITECTURE**

The system follows a three-tier architecture:

Frontend: HTML, CSS

Backend: PHP

Database: MySQL

Working Flow:

User → Login Page → PHP Processing → MySQL

Database → Output Display

# DATABASE DESIGN

Users user\_id (INT, Primary Key, Auto Increment) name (VARCHAR(100)) email (VARCHAR(100)) role (VARCHAR(50))

Requests request\_id (INT, Primary Key, Auto Increment) user\_id (INT, Foreign Key → Users.user\_id) category\_id (INT, Foreign Key → Categories.category\_id) title (VARCHAR(150)) description (TEXT) created\_at (DATETIME)

Categories category\_id (INT, Primary Key, Auto Increment) category\_name (VARCHAR(100)) description (TEXT)

Status status\_id (INT, Primary Key, Auto Increment) status\_name (VARCHAR(50))

Assignments assignment\_id (INT, Primary Key, Auto Increment) request\_id (INT, Foreign Key →

Requests.request\_id) assigned\_to (INT, Foreign Key  
→ Users.user\_id) assigned\_date (DATETIME)

Request\_History history\_id (INT, Primary Key, Auto  
Increment) request\_id (INT, Foreign Key →  
Requests.request\_id) status\_id (INT, Foreign Key →  
Status.status\_id) updated\_at (DATETIME) remarks  
(TEXT)

# FUNCTIONAL MODULES

## 1. Authentication Module

login.php register.php  
logout.php

## 2. Request Module

submit\_request.php  
my\_requests.php  
request\_details.php

## 3. Admin Module

dashboard.php  
manage\_requests.php

## 4. Assignment Module

assign\_request.php

## 5. Status Module

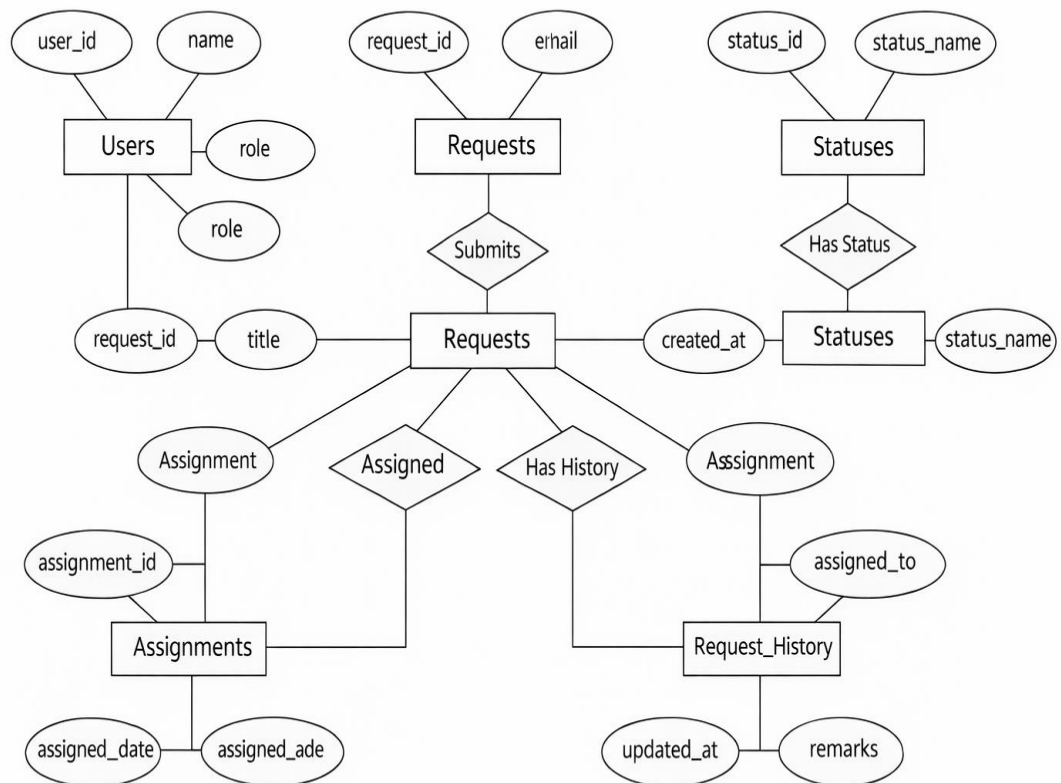
update\_status.php

## 6. Database Module

db.php

## 7. UI Module style.css

# ER DIAGRAM



The ER diagram represents a Complaint Management System where users submit requests, requests are categorized, assigned to staff, and their status is tracked through history records

# SQL QUERIES

```
CREATE TABLE Users (  user_id INT PRIMARY  
KEY AUTO_INCREMENT,  name  
VARCHAR(100),  email VARCHAR(100),  role  
VARCHAR(50)  
);
```

```
CREATE TABLE Request_History (  history_id INT  
PRIMARY KEY AUTO_INCREMENT,  request_id INT,  
status_id INT,  updated_at DATETIME,  remarks TEXT,  
  FOREIGN      KEY      (request_id)      REFERENCES  
Requests(request_id),  
  FOREIGN KEY (status_id) REFERENCES Status(status_id)  
);
```

```
CREATE TABLE Assignments (  assignment_id INT  
PRIMARY KEY AUTO_INCREMENT,  request_id INT,  
assigned_to INT,  
  assigned_date DATETIME,  
  FOREIGN      KEY      (request_id)      REFERENCES  
Requests(request_id),  
  FOREIGN KEY (assigned_to) REFERENCES Users(user_id)
```

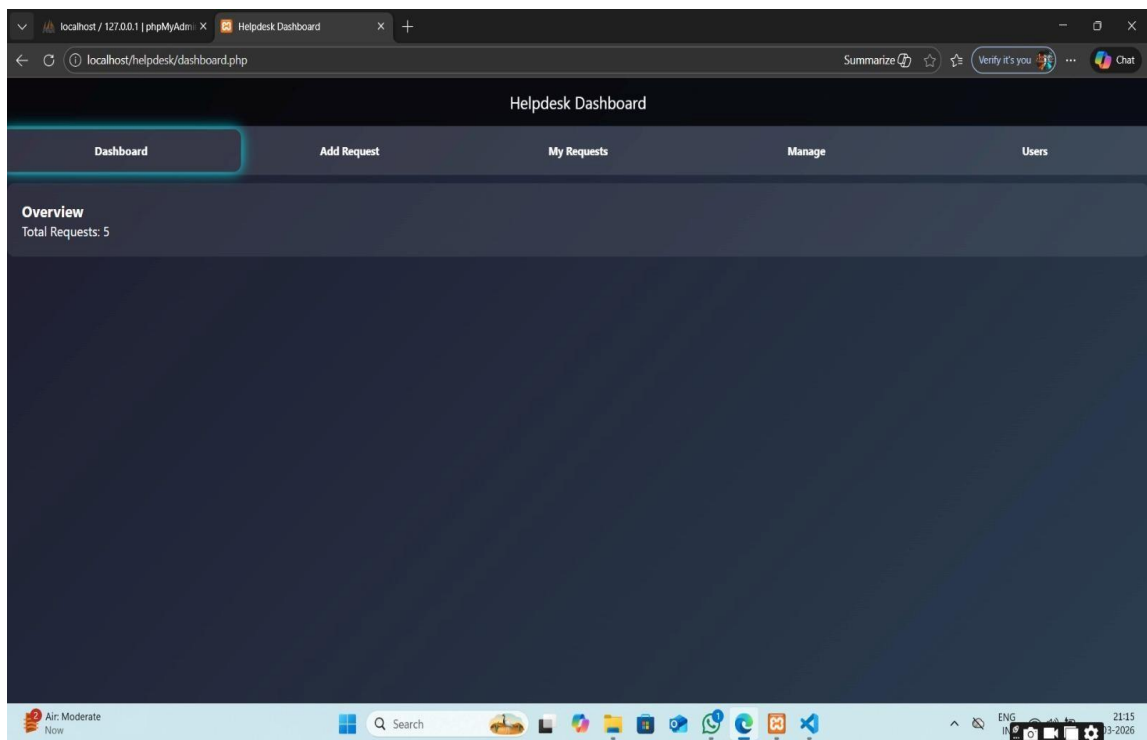
);

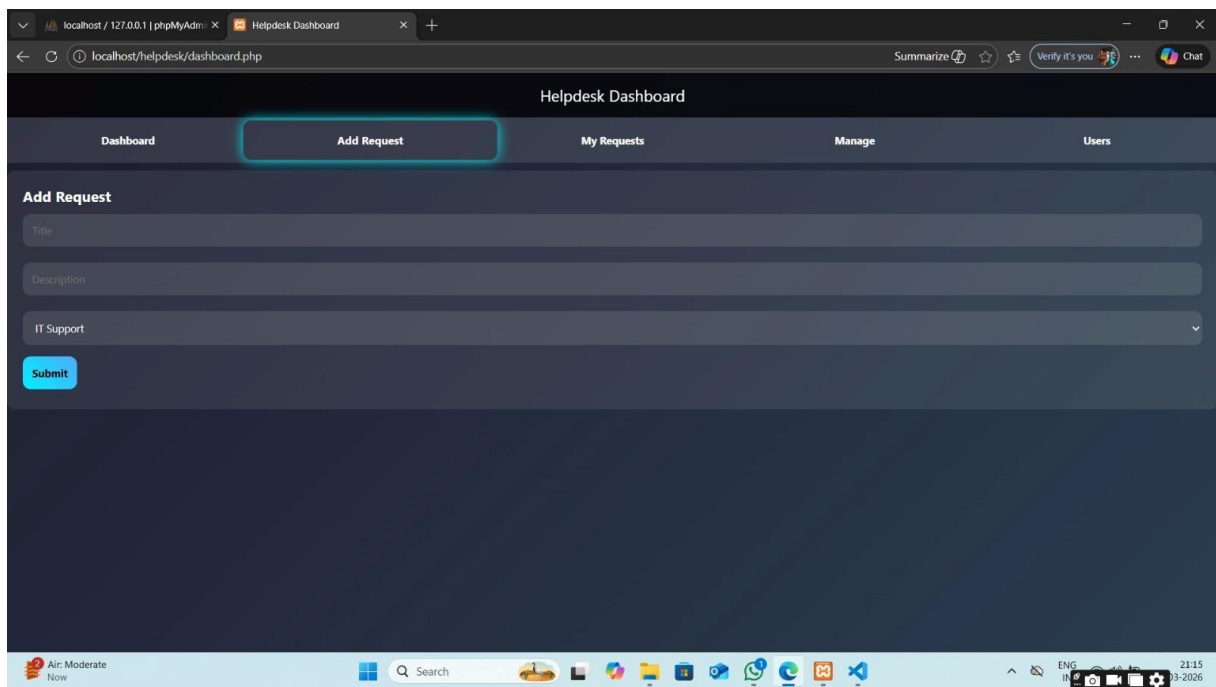
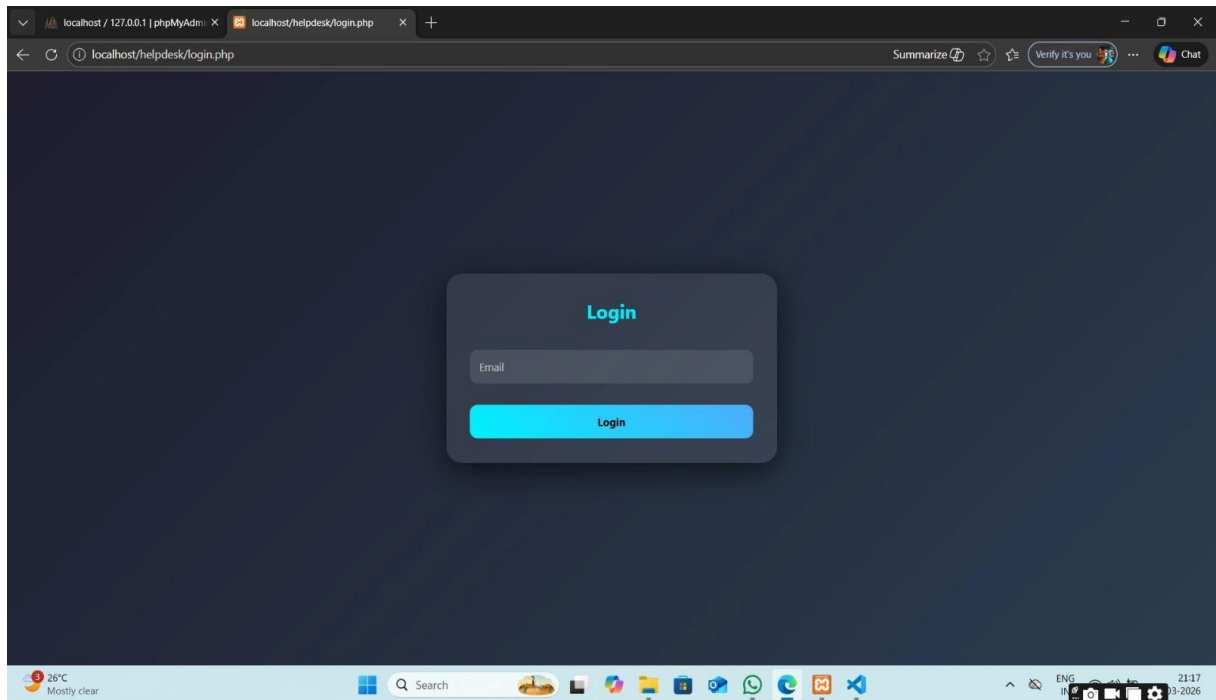
```
CREATE TABLE Status (    status_id INT PRIMARY
KEY AUTO_INCREMENT,      status_name
VARCHAR(50)
);
```

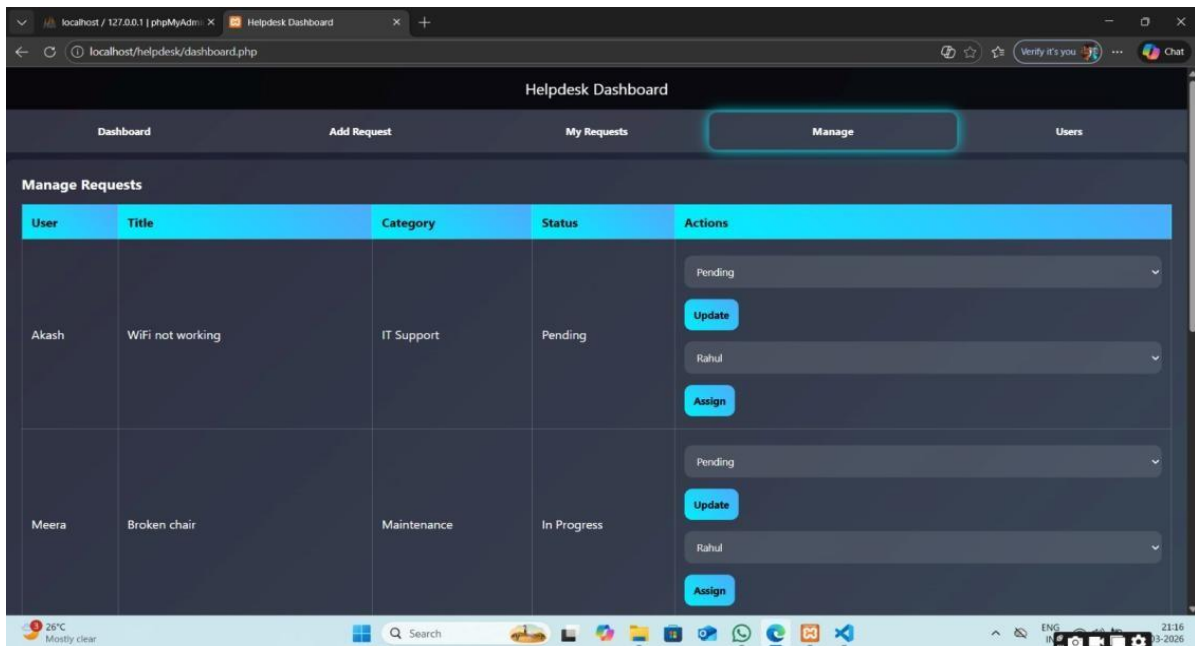
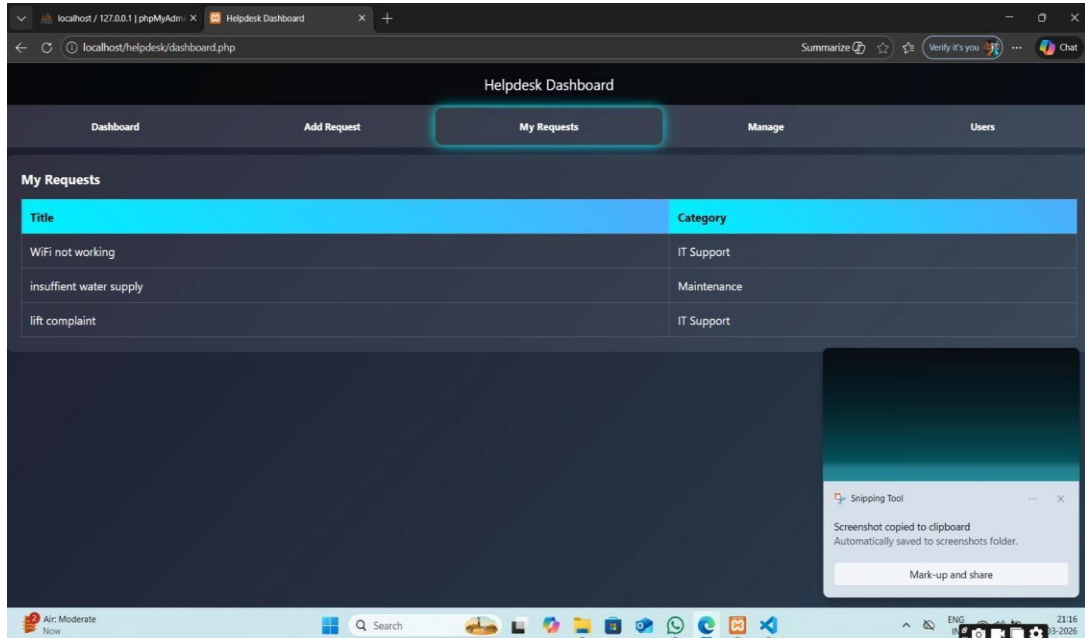
```
CREATE TABLE Requests (    request_id INT PRIMARY
KEY AUTO_INCREMENT,    user_id INT,    category_id
INT,    title VARCHAR(150),    description TEXT,
created_at DATETIME,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (category_id) REFERENCES
Categories(category_id)
);
```

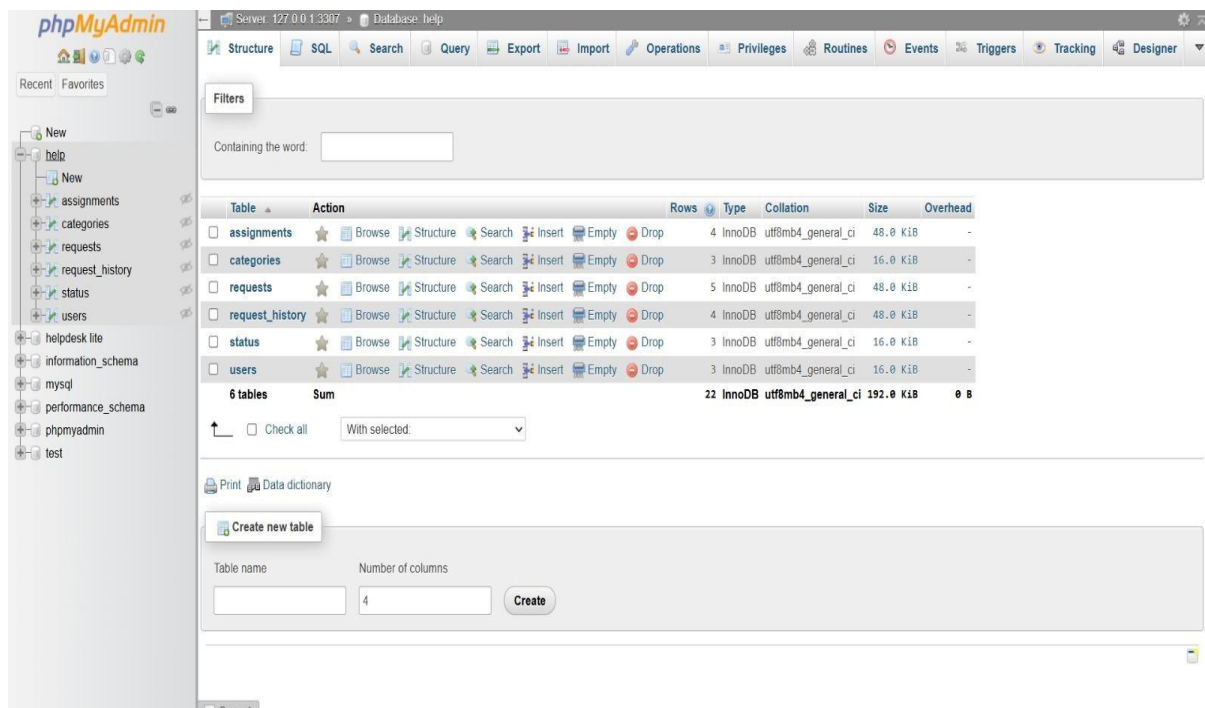
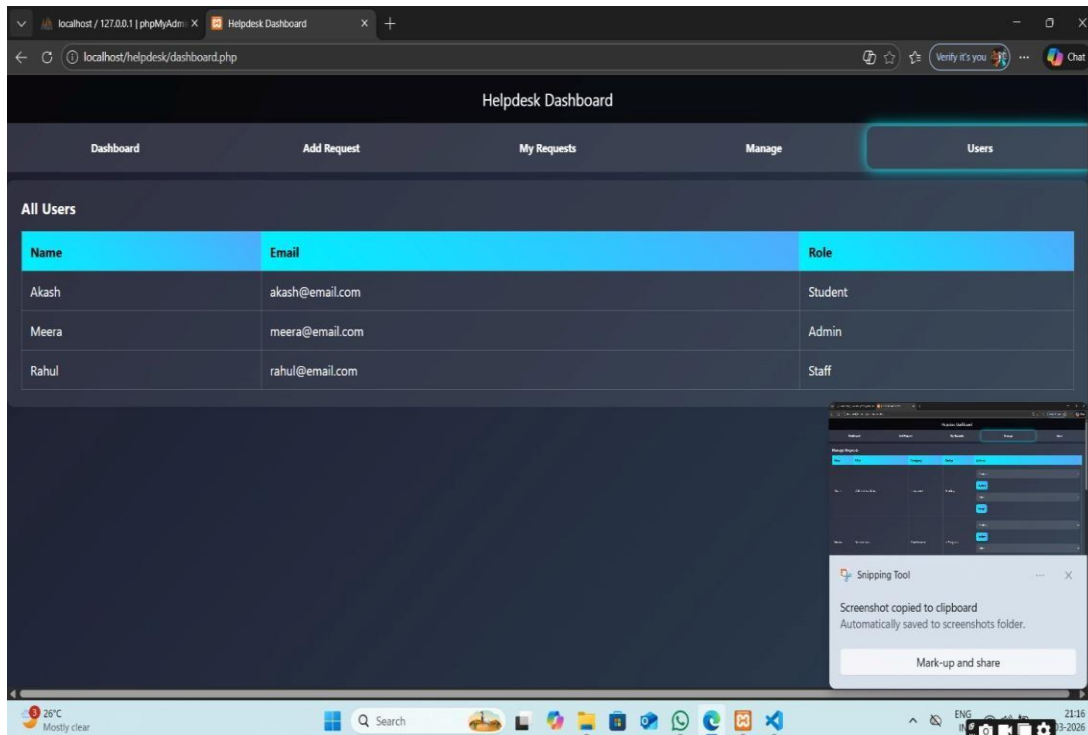
```
CREATE TABLE Categories (    category_id INT
PRIMARY KEY AUTO_INCREMENT,    category_name
VARCHAR(100),    description TEXT
);
```

# OUTPUT









# LIMITATIONS

The system supports only **basic authentication** and does not include advanced security features like encryption or multifactor authentication

- It currently lacks **role-based access control**, so user permissions (Admin, Staff, User) are not strictly enforced
- There is no **real-time notification system** (such as email or SMS alerts) for status updates
- The system is designed for **small-scale environments** and may not handle large-scale data or high user traffic efficiently
- Limited **user interface design**, focusing more on functionality than advanced user experience
- No **analytics or reporting module** to track performance metrics like resolution time or request trends
- The system depends on **manual assignment of requests**, which may lead to delays or inefficiencies
- There is no **file upload or attachment feature** for users to provide additional proof (images/documents)
- The application does not support **mobile responsiveness or cross-platform optimization**
- Backup and recovery mechanisms are **not implemented**, which may risk data loss

## FUTURE SCOPE

- Implementation of **role-based access control** to clearly define permissions for Admin, Staff, and Users
- Integration of **real-time notification systems** (Email/SMS) for request updates and alerts
- Development of an **analytics dashboard** to monitor request trends, resolution time, and system performance
- Addition of **file upload functionality** to allow users to attach images or documents with requests
- Automation of request assignment using **priority-based or rule-based allocation systems**
- Enhancement of the user interface with a more **responsive and mobile-friendly design**
- Integration with **mobile applications** (Android/iOS) for easier accessibility
- Implementation of **advanced security features** such as encryption and secure authentication mechanisms
- Addition of **search and filter options** for efficient request management
- Incorporation of **backup and recovery systems** to ensure data safety and reliability

# CONCLUSION

HelpDesk Lite successfully demonstrates the design and implementation of a **database-driven service request management system**. The project provides a structured and centralized platform for users to submit complaints, track their progress, and receive timely updates, replacing traditional unorganized methods.

Through this system, core **DBMS concepts** such as relational schema design, primary and foreign key constraints, normalization, and data integrity have been effectively applied. The integration of modules like request handling, assignment, and status tracking ensures a complete lifecycle management of each request.

The use of dedicated tables such as **Requests, Assignments, Status, and Request\_History** enables efficient data management and maintains transparency and accountability throughout the process.

Although the system currently focuses on essential functionalities, it has strong potential for future enhancements such as role-based access control, notifications, and analytics.

In conclusion, HelpDesk Lite highlights how **database management systems can be utilized to build practical, scalable solutions for real-world service coordination problems**, improving efficiency and user experience